

Name of the Student	T.Chaitra
Reg. No	192425217
GitHub Link	https://github.com/chaitramanohar/MLA0402-DEEP-LEARNING

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Model

COURSE NAME: Deep Learning
SLOT :

COURSE CODE: MLA04

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10%
Assessment Tool Weightage: 30%

Duration: 90 minutes
Mark : 25

Code of Conduct:

I T.CHAITRA(192425217)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:T.Chaitra

Name of the student:T.Chaitra

Criteria	Problem Understanding & Requirement Analysis (2)	Model Architecture & Design (2.5)	Application of Deep Learning Techniques (2.5)	Model Development & Implementation Strategy (2)	Performance Analysis & Justification (1)	Total (10)
Weightage	20 %	30%	25%	15 %	10 %	100 %
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

QUESTIONS:05

Total Marks:25

- 1. Desing and develop a model for a** hospital has only **2,000 labeled X-ray images** for classifying images into normal and abnormal categories. A pre-trained ResNet model is available. **Design a CNN-based transfer learning model** by identifying which layers should be frozen, which layer should be replaced, and how the model should be trained.
- 2.** An agricultural organization wants to identify **five types of plant diseases** from leaf images. Since the dataset contains limited training samples, the organization wants to reuse features efficiently. **Design a DenseNet-based classification model** and explain how dense connections can help the model reuse features during classification.
- 3.** A university wants to predict a student's future academic performance using their **weekly attendance, assignment marks, internal assessment marks, and previous examination scores** collected over several weeks. **Design an LSTM-based model** that can process this sequential information and predict the student's performance category.
- 4.** An organization wants to classify customer reviews as **positive, negative, or neutral**. Since the meaning of a word may depend on both the words appearing before and after it, **design a Bidirectional RNN model** for this application. Identify the major processing stages from input text to final sentiment prediction.
- 5.** A software company wants to develop a system that translates **English sentences into Hindi** . The length of the input and output sentences may differ, and the system needs to understand the complete input sentence before generating the translated sentence. **Design an Encoder–Decoder model** and describe the role of the encoder, decoder, hidden representation, and output layer in the translation process.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Requirement Analysis	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding ; some requirements unclear	Poor understanding ; misinterprets problem context
Model Architecture & Design	30%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Application of Deep Learning Techniques	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Model Development & Implementation Strategy	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Performance Analysis & Justification	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

1. Transfer Learning using Pre-trained ResNet for X-ray Classification

Scenario

A hospital has only **2,000 labeled X-ray images** classified into **Normal** and **Abnormal**. Since the dataset is small, training a CNN from scratch may cause overfitting. Therefore, a **pre-trained ResNet** can be used with transfer learning.

Proposed Architecture

Input X-ray → ResNet → Global Average Pooling → Dense Layer → Output

X-ray Image



Resize + Normalize



Pre-trained ResNet



Global Average Pooling



Dense Layer + ReLU



Dropout



Dense(1) + Sigmoid



Normal / Abnormal

Which layers should be frozen?

Initially, **freeze most/all convolutional layers** of the pre-trained ResNet.

These layers have already learned general features such as:

- Edges
- Shapes
- Textures
- Patterns

Only the newly added classification layers are trained initially.

Which layer should be replaced?

The original final classification layer of ResNet is designed for its original dataset/classes.

Therefore:

Remove the original fully connected classification layer and replace it with:

Global Average Pooling

↓
Dense(128, ReLU)
↓
Dropout
↓
Dense(1, Sigmoid)

Training Process

Step 1: Load pre-trained ResNet.

Step 2: Remove the original classification layer.

Step 3: Add a binary classification head.

Step 4: Freeze the ResNet convolutional layers.

Step 5: Train only the new layers.

Step 6: After initial training, optionally **unfreeze the last few ResNet layers** and perform fine-tuning using a **small learning rate**.

Loss Function

Since there are two classes:

Binary Cross-Entropy

Advantages

- Requires less training data.
- Reduces training time.
- Reuses previously learned features.
- Helps reduce overfitting.
- Fine-tuning improves medical-image-specific performance.

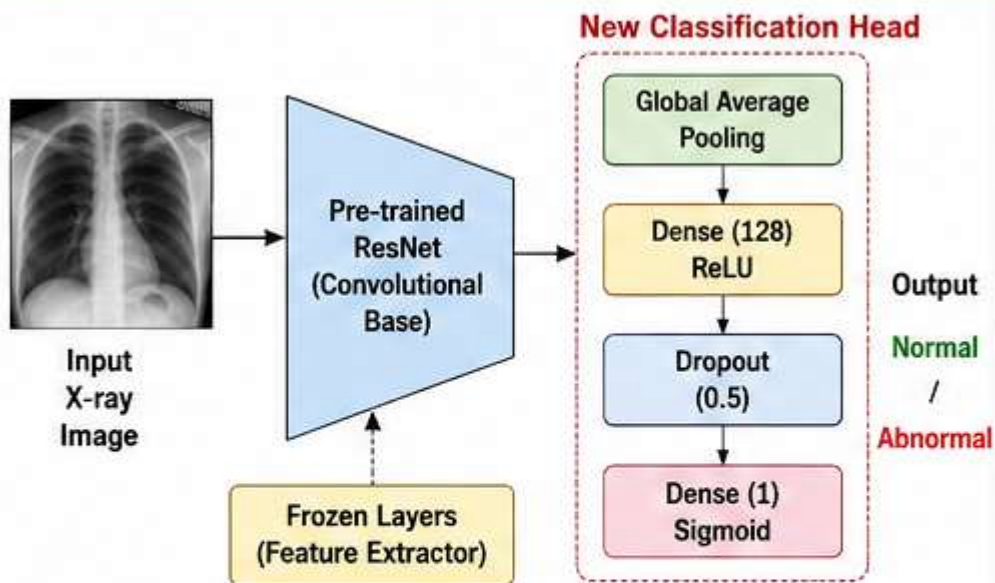
Exam conclusion

A pre-trained ResNet with frozen convolutional layers and a newly added binary classification head is suitable for the small X-ray dataset. Initially only the new layers are trained, followed by optional fine-tuning of the deeper ResNet layers using a low learning rate.

Memory line:

Freeze → Replace → Train → Fine-tune

1. Transfer Learning using ResNet for X-ray Classification



- Freeze all convolutional layers of pre-trained ResNet.
- Replace the original classification layer with a new binary classification head.
- Train only the new layers, then fine-tune some deeper layers.

2. DenseNet for Plant Disease Classification

Scenario

An agricultural organization wants to classify **five plant diseases** from leaf images. Since training samples are limited, a **DenseNet** model can efficiently reuse previously learned features.

Proposed Architecture

Leaf Image



Resize + Normalization



Pre-trained DenseNet



Dense Blocks



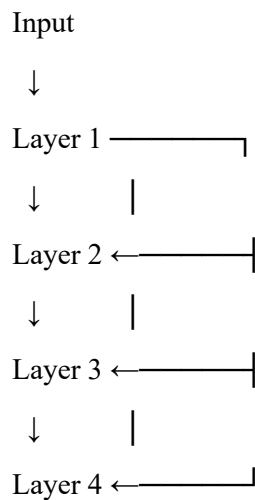
Global Average Pooling

↓
Dense Layer
↓
Softmax
↓
5 Disease Classes

What is DenseNet?

DenseNet is a CNN architecture where **each layer receives the outputs of all previous layers within a dense block**.

For example:



Mathematically:

$$x_l = H_l([x_0, x_1, x_2, \dots, x_{l-1}])$$

where [] represents concatenation of previous feature maps.

How does DenseNet reuse features?

Suppose an early layer detects:

- Leaf edges
- Color patterns
- Veins

Later layers can directly access these features instead of learning them again.

Therefore, DenseNet provides:

1. **Feature reuse**
2. Better gradient flow
3. Reduced redundant feature learning
4. Fewer parameters compared with some traditional CNNs
5. Better performance with limited training data

Classification Layer

Since there are **5 disease classes**, the final layer is:

Dense(5, activation='softmax')

The class with the highest probability is selected.

Example:

Disease 1 $\rightarrow$ 0.05

Disease 2 $\rightarrow$ 0.10

Disease 3 $\rightarrow$ 0.70 $\leftarrow$ Prediction

Disease 4 $\rightarrow$ 0.10

Disease 5 $\rightarrow$ 0.05

Training

- Use a pre-trained DenseNet.
- Freeze initial layers.
- Replace the final classifier with a 5-class classifier.
- Train the new classification layer.
- Fine-tune some deeper layers with a low learning rate.

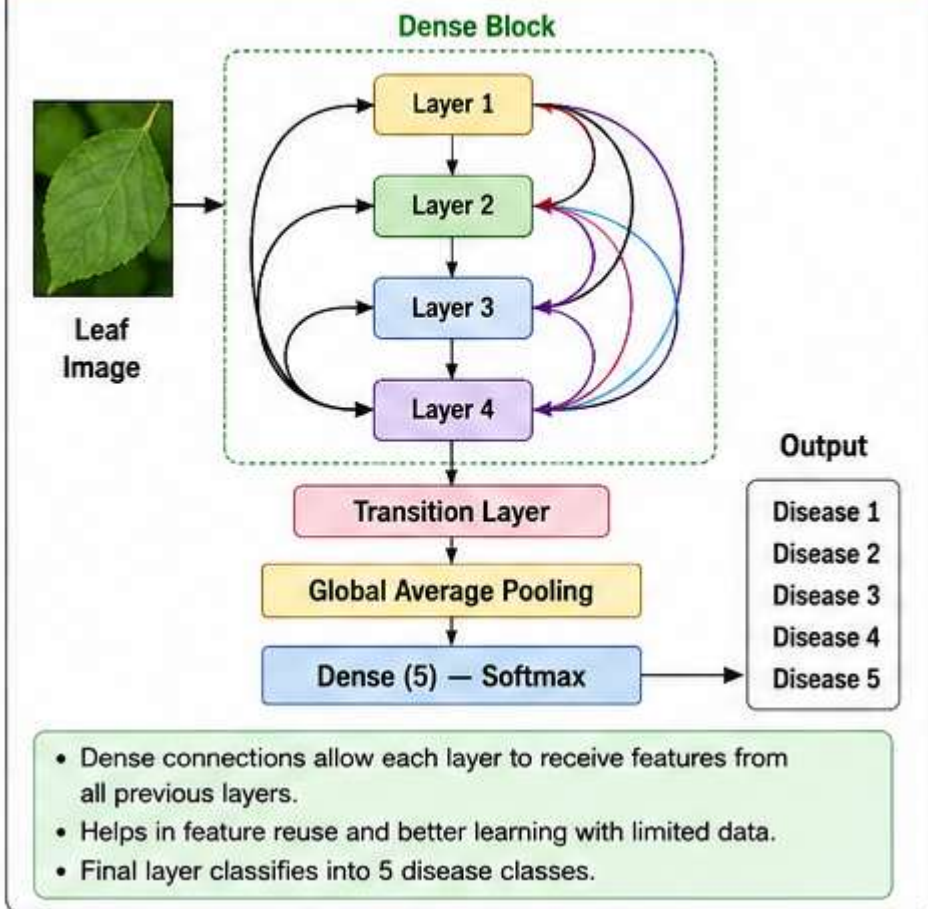
Conclusion

DenseNet is suitable for plant disease classification because dense connections allow the network to reuse features from earlier layers. This is particularly useful when the available training dataset is small.

Memory line:

DenseNet = Every layer connects to previous layers $\rightarrow$ Feature reuse

2. DenseNet for Plant Disease Classification



3. LSTM for Student Academic Performance Prediction

Scenario

The university has student information collected **week by week**:

- Attendance
- Assignment marks
- Internal assessment marks
- Previous examination scores

Since this is **sequential/time-series data**, LSTM is suitable.

Proposed Architecture

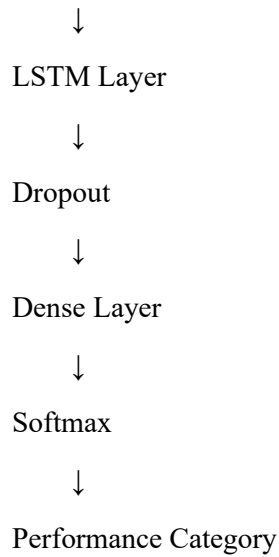
Weekly Student Data

↓

Data Preprocessing

↓

Sequence Formation



Input Representation

For each week:

$$X_t = [A, t, t, endanceAssignmentInternalExamScore]$$

For example:

Week 1 → [90, 80, 75, 70]

Week 2 → [85, 82, 78, 72]

Week 3 → [92, 88, 80, 78]

Week 4 → [95, 90, 85, 82]

The LSTM processes these values **in sequence**.

Why LSTM?

LSTM is designed to remember important information from previous time steps.

For example:

Low attendance during Week 2 may affect performance later.

LSTM can capture such temporal relationships.

Architecture

Example:

Input Shape

(time steps, 4 features)

↓
LSTM(64)

↓
Dropout

↓

Dense(32, ReLU)



Dense(3, Softmax)

The three categories could be:

- Poor
- Average
- Excellent

Working

At every time step, the LSTM uses:

- Previous hidden state
- Current input
- Cell state

The **cell state** helps preserve important information over multiple weeks.

Output

The final Softmax layer gives probabilities:

Poor → 0.10

Average → 0.25

Excellent → 0.65

Therefore:

Prediction = Excellent

Loss Function

For three performance categories:

Categorical Cross-Entropy

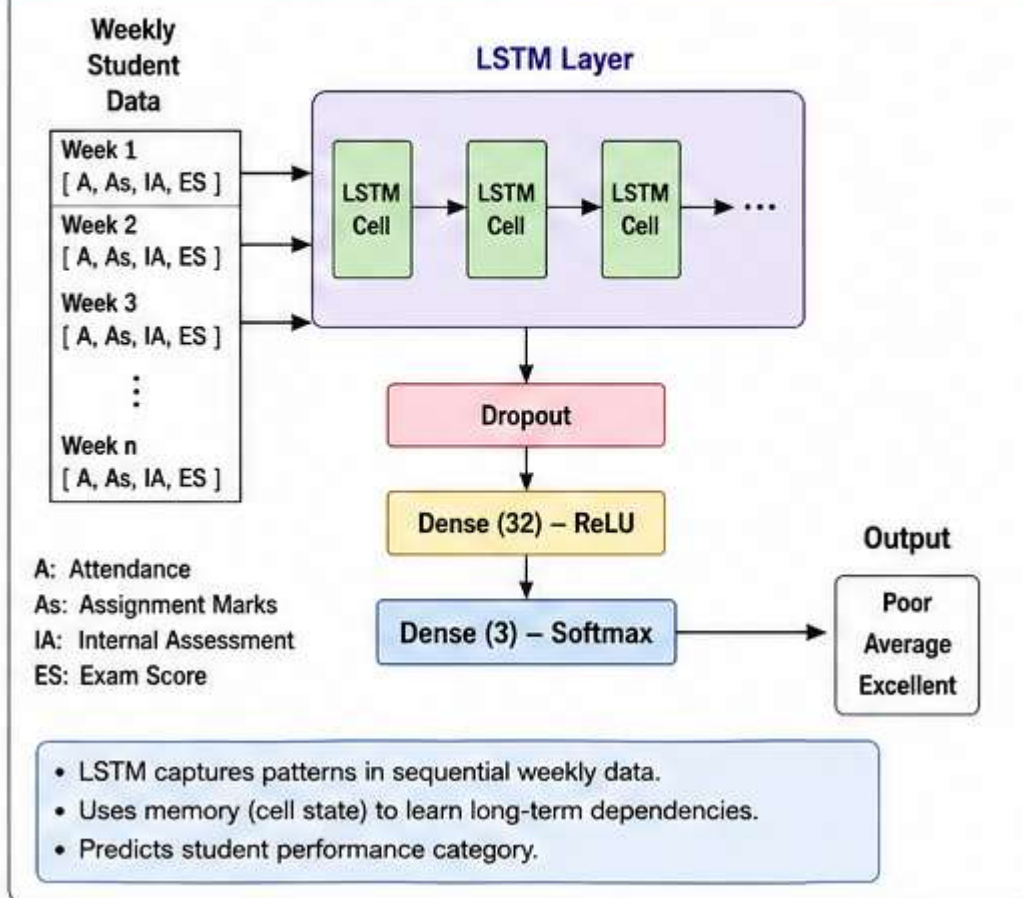
Conclusion

LSTM is suitable because student academic data is sequential. It can learn relationships between performance measurements across multiple weeks and predict the student's future performance category.

Memory line:

LSTM = Sequence + Memory → Future prediction

3. LSTM for Student Academic Performance Prediction



4. Bidirectional RNN for Customer Sentiment Classification

Scenario

Customer reviews need to be classified as:

- Positive
- Negative
- Neutral

The meaning of a word can depend on both **previous and following words**.

Therefore, a **Bidirectional RNN** is suitable.

Example

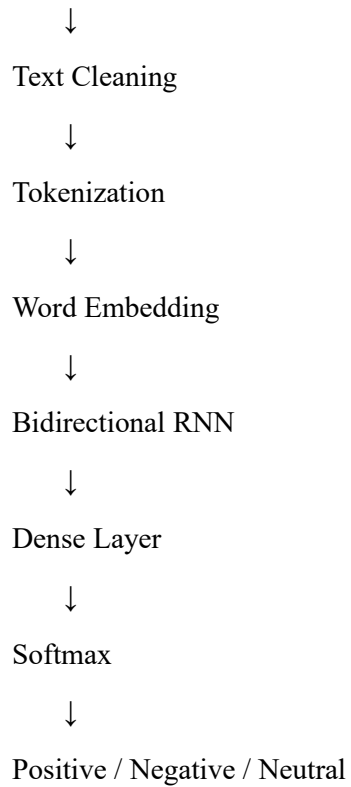
Consider:

"The movie was not very good."

The word **good** needs surrounding context to correctly understand the sentiment.

Proposed Architecture

Customer Review



Major Processing Stages

1. Text Preprocessing

Clean the review:

- Convert to lowercase
- Remove unnecessary symbols
- Tokenize words

Example:

"The product is very good!"



["the", "product", "is", "very", "good"]

2. Tokenization

Each word is converted into a numerical token.

the → 12

product → 45

good → 78

3. Word Embedding

Token numbers are converted into dense vectors.

Token → Embedding Vector

4. Bidirectional RNN

Two RNNs process the sentence:

Forward RNN:

The → product → is → very → good

Backward RNN:

good → very → is → product → The

Their outputs are combined.

Forward information

+

Backward information

↓

Combined representation

5. Dense Layer

The combined representation is passed to a fully connected layer.

6. Softmax Output

Positive → 0.80

Negative → 0.10

Neutral → 0.10

Prediction:

Positive

Why Bidirectional RNN?

A normal RNN mainly processes information in one direction.

A Bidirectional RNN considers:

Past context + Future context

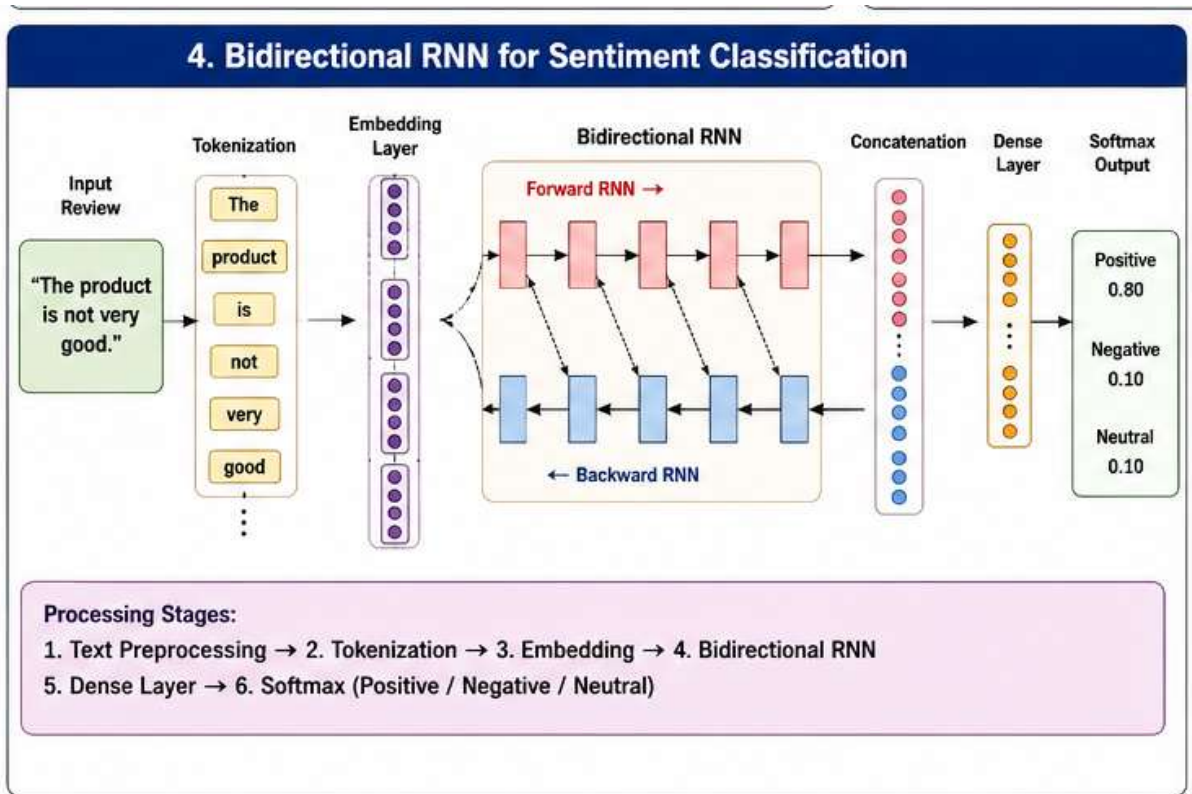
Therefore, it can understand the meaning of words more effectively.

Conclusion

A Bidirectional RNN is appropriate for sentiment analysis because it processes the review in both forward and backward directions, allowing the model to use context from both sides of a word.

Memory line:

Bi-RNN = Past context + Future context



5. Encoder–Decoder Model for English → Hindi Translation

Scenario

A software company wants to translate English sentences into Hindi.

Example:

English:

"I am going to school."

Hindi:

"मैं स्कूल जा रहा हूँ।"

The input and output sentences may have **different lengths**, so an Encoder–Decoder architecture is suitable.

Architecture

English Sentence

↓

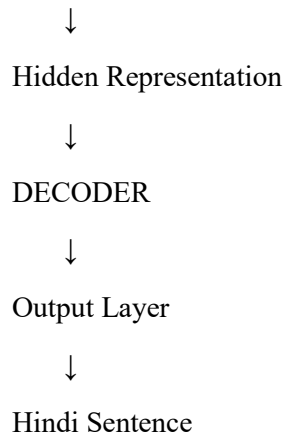
Tokenization

↓

Embedding

↓

ENCODER



1. Input / Embedding

The English sentence is divided into tokens.

I | am | going | to | school

Each token is converted into an embedding vector.

2. Encoder

The encoder processes the complete input sequence.

Its job is to understand the meaning and context of the English sentence.

For example:

I → am → going → to → school

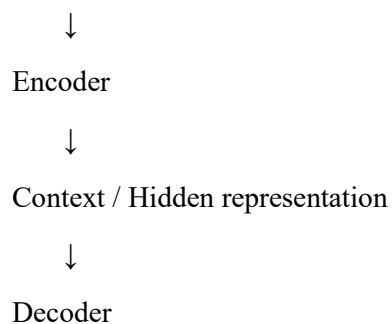
The encoder generates a **hidden representation/context representation** containing information about the input sentence.

3. Hidden Representation

The hidden representation acts as a summary of the input sentence.

It transfers the information learned by the encoder to the decoder.

English sentence



4. Decoder

The decoder generates the Hindi translation **one word at a time**.

For example:

<START>



मैं
↓
स्कूल
↓
जा
↓
रहा
↓
हूँ
↓
<END>

The decoder uses:

- Hidden state
- Previous generated word
- Context information

to predict the next word.

5. Output Layer

The output layer uses **Softmax** to calculate the probability of every Hindi word in the vocabulary.

Example:

मैं → 0.70

वह → 0.10

स्कूल → 0.05

जा → 0.15

The highest-probability word is selected.

Why Encoder–Decoder?

The model can handle:

- Different input/output sentence lengths
- Sequence-to-sequence translation
- Context-dependent language generation

Complete Flow

English

"I am going to school"

↓

Encoder



Hidden Representation



Decoder



"मैं स्कूल जा रहा हूँ"

Conclusion

The Encoder–Decoder model first converts the complete English sentence into a hidden representation. The decoder then uses this representation to generate the Hindi sentence sequentially, while the output Softmax layer predicts the probability of each Hindi word.

Memory line:

Encoder = Understand → Hidden state = Store meaning → Decoder = Generate

